



EC200D&ECx00G&EC600U&EG800G Series

QuecOpen Data Call API Reference Manual

LTE Standard Module Series

Version: 1.0

Date: 2023-08-31

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2022-01-06	Braden HE/ Morgan ZHU	Creation of the document
1.0	2023-08-31	Joe TU	First official release

Contents

About the Document.....	3
Contents	4
Table Index.....	5
Figure Index	6
1 Introduction	7
1.1. Applicable Modules	7
2 API Calling Process	8
3 Data Call API.....	10
3.1. Header File	10
3.2. API Overview	10
3.3. API Description.....	11
3.3.1. ql_network_register_wait.....	11
3.3.1.1. ql_datacall_errcode_e	11
3.3.2. ql_set_data_call_asyn_mode	13
3.3.3. ql_start_data_call.....	14
3.3.3.1. ql_pdp_auth_type_e	16
3.3.4. ql_stop_data_call.....	16
3.3.5. ql_get_data_call_info	17
3.3.5.1. ql_data_call_info_s.....	18
3.3.5.2. struct v4_info	18
3.3.5.3. struct v4_address_status	19
3.3.5.3.1. ip4_addr_t	19
3.3.5.4. struct v6_info	19
3.3.5.5. struct v6_address_status	20
3.3.5.5.1. ip6_addr_t	20
3.3.6. ql_datacall_register_cb.....	21
3.3.6.1. ql_datacall_callback	21
3.3.7. ql_datacall_unregister_cb.....	22
3.3.8. ql_datacall_get_sim_profile_is_active	23
3.3.9. ql_bind_sim_and_profile.....	24
3.3.10. ql_get_sim_and_profile.....	24
3.3.11. ql_datacall_set_nat.....	25
3.3.12. ql_datacall_get_nat.....	25
3.3.13. ql_datacall_set_default_pdn_cfg	26
3.3.13.1. ql_data_call_default_pdn_cfg_s.....	26
3.3.14. ql_datacall_get_default_pdn_cfg.....	27
3.3.15. ql_datacall_get_default_pdn_info.....	27
3.3.15.1. ql_data_call_default_pdn_info_s.....	28
4 Appendix References	29

Table Index

Table 1: Applicable Modules..... 7

Table 2: API Overview 10

Table 3: Related Documents..... 29

Table 4: Terms and Abbreviations 29

Figure Index

Figure 1: Data Call Process 8

Figure 2: Auto-Start Data Call Example Program 9

1 Introduction

Quectel EC200D series, EC600U series, EG800G series and ECx00G family modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]** and **document [2]**.

This document explains data call, including API calling process and provides API description on Quectel EC200D series, EC600U series, EG800G series and ECx00G family modules in QuecOpen® solution.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC200D Series
-	EC600U Series
ECx00G	EC200G-CN
	EG600G Series
	EC800G-CN
-	EG800G Series

NOTE

For the EC200D series module, the file paths mentioned in this document do not contain *components*.

2 API Calling Process

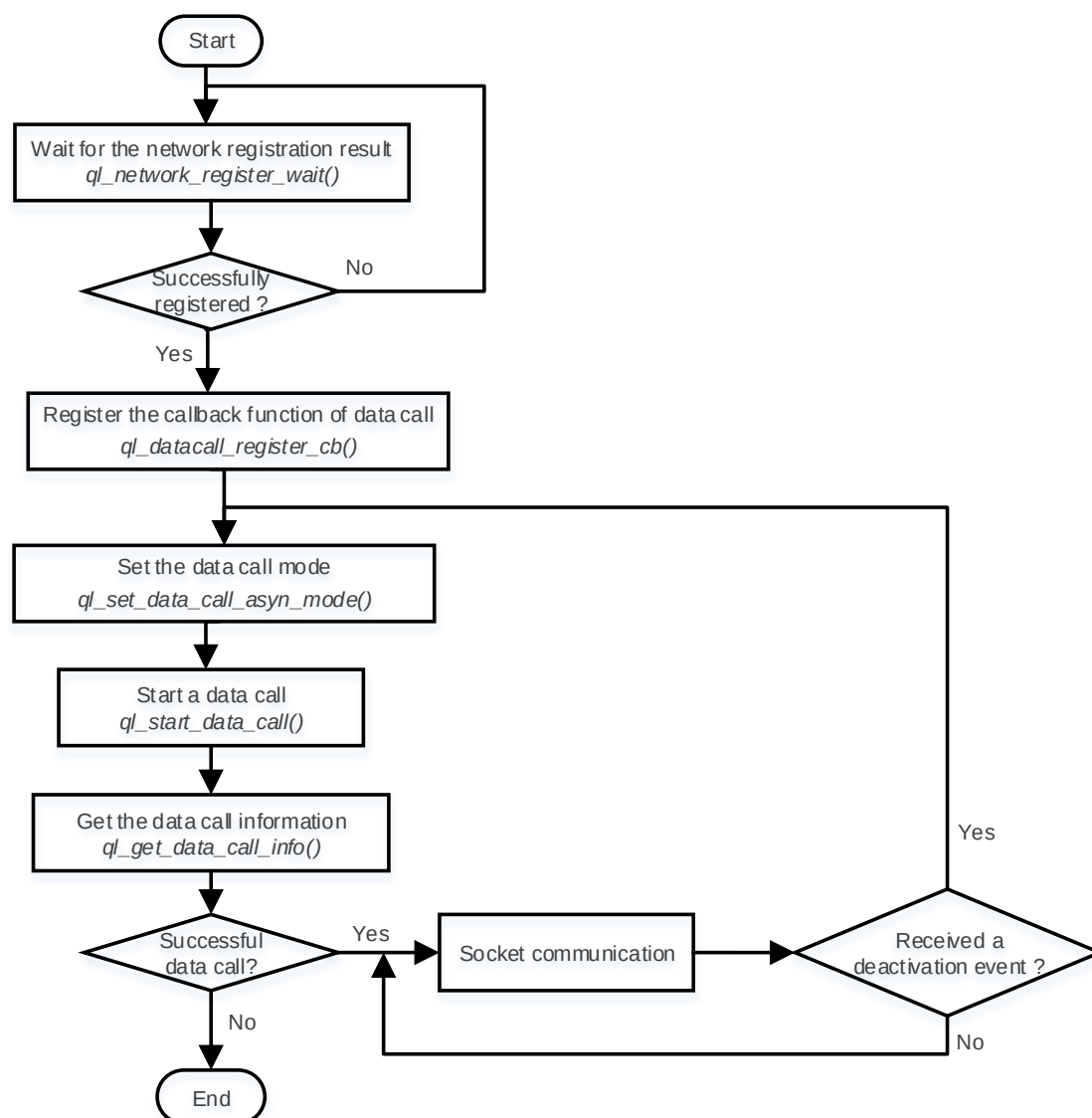


Figure 1: Data Call Process

Data call process includes three parts: network registration, data call and socket establishment.

Network registration is performed automatically and requires no code intervention. You only need to call *ql_network_register_wait()* to wait for the registration to complete. If the current module does not register network, the thread currently calling this function will be blocked until the network registration is successful

or times out. You can also call `ql_nw_register_cb()` to register the callback function for network event to receive `QUEC_NW_DATA_REG_STATUS_IND` event. See **document [3]** for detailed information about `ql_nw_register_cb()`.

PDP context activation can be carried out once the module has successfully registered the network. Current data call supports both synchronous and asynchronous modes, which can be set with `ql_set_data_call_asyn_mode()`. See **Chapter 3.3.2** for detailed information.

Socket communication can be carried out after a successful data call. To help you understand the data call process, `datacall_demo.c`, the example program, is available in `components\ql-application\nw\` directory of QuecOpen SDK.

To start the example program automatically, uncomment `ql_datacall_app_init()` under `ql_init_demo_thread()` in `components\ql-application\init\ql_init.c` file, then compile the firmware version and download it to the module. When the module boots, the example program starts and automatically performs a data call.

```

04:
05: static void ql_init_demo_thread(void *param)
06: {
07:     QL_INIT_LOG("init demo thread enter, param 0x%x", param);
08:
09:     ql_gpio_app_init();
10:     ql_gpioint_app_init();
11:     ql_ledcfg_app_init();
12:
13:     #ifdef QL_APP_FEATURE_AUDIO
14:         ql_audio_app_init();
15:     #endif
16:     #ifdef QL_APP_FEATURE_LCD
17:         ql_lcd_app_init();
18:     #endif
19:     ql_nw_app_init();
20:     //ql_datacall_app_init();
21:     ql_osi_demo_init();
22:
23:     #ifdef QL_APP_FEATURE_FILE
24:         ql_fs_demo_init();
25:     #endif
26:

```

Figure 2: Auto-Start Data Call Example Program

3 Data Call API

This chapter introduces data call API, enumerations and structures.

3.1. Header File

ql_api_datacall.h, the header file of data call API, is in *components\ql-kernel\inc* directory. Unless otherwise specified, the header files mentioned in this document are all in this directory.

3.2. API Overview

Table 2: API Overview

Function	Description
<i>ql_network_register_wait()</i>	Waits for the network registration result.
<i>ql_set_data_call_asyn_mode()</i>	Sets the execution mode of <i>ql_start_data_call()</i> and <i>ql_stop_data_call()</i> .
<i>ql_start_data_call()</i>	Starts a data call.
<i>ql_stop_data_call()</i>	Stops a data call.
<i>ql_get_data_call_info()</i>	Gets data call information.
<i>ql_datacall_register_cb()</i>	Registers the callback function of data call.
<i>ql_datacall_unregister_cb()</i>	Cancels the callback function registered with <i>ql_datacall_register_cb()</i> .
<i>ql_datacall_get_sim_profile_is_active()</i>	Gets the current status of PDP context activation.
<i>ql_bind_sim_and_profile()</i>	Binds the (U)SIM card in use and PDP context ID, and gets <i>sim_cid</i> , the combination of (U)SIM card and PDP context ID.
<i>ql_get_sim_and_profile()</i>	Gets (U)SIM card and PDP context ID from <i>sim_cid</i> .

<code>ql_datacall_set_nat()</code>	Enables NAT.
<code>ql_datacall_get_nat()</code>	Gets the NAT-enabled (U)SIM card and PDP context ID.
<code>ql_datacall_set_default_pdn_cfg()</code>	Sets the default bearer configuration
<code>ql_datacall_get_default_pdn_cfg()</code>	Gets the default bearer configuration.
<code>ql_datacall_get_default_pdn_info()</code>	Gets the default bearer information.

3.3. API Description

3.3.1. ql_network_register_wait

This function waits for the network registration result.

● Prototype

```
ql_datacall_errcode_e ql_network_register_wait(uint8_t nSim, unsigned int timeout_s)
```

● Parameter

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

timeout_s:

[In] Waiting for network registration timeout. Unit: second.

● Return Value

See **Chapter 3.3.1.1** for detailed information.

3.3.1.1. ql_datacall_errcode_e

The data call result codes indicate whether the function is executed successfully. The enumeration of result codes is defined as below:

```
typedef enum
{
    QL_DATACALL_SUCCESS                = 0,
    QL_DATACALL_EXECUTE_ERR            = 1 | QL_DATACALL_ERRCODE_BASE,
    QL_DATACALL_MEM_ADDR_NULL_ERR,
```

```

QL_DATACALL_INVALID_PARAM_ERR,
QL_DATACALL_NW_REGISTER_TIMEOUT_ERR,
QL_DATACALL_CFW_ACT_STATE_GET_ERR    = 5 | QL_DATACALL_ERRCODE_BASE,
QL_DATACALL_REPEAT_ACTIVE_ERR,
QL_DATACALL_REPEAT_DEACTIVE_ERR,
QL_DATACALL_CFW_PDP_CTX_SET_ERR,
QL_DATACALL_CFW_PDP_CTX_GET_ERR,
QL_DATACALL_CS_CALL_ERR                = 10 | QL_DATACALL_ERRCODE_BASE,
QL_DATACALL_CFW_CFUN_GET_ERR,
QL_DATACALL_CFUN_DISABLE_ERR,
QL_DATACALL_NW_STATUS_GET_ERR,
QL_DATACALL_NOT_REGISTERED_ERR,
QL_DATACALL_NO_MEM_ERR                  = 15 | QL_DATACALL_ERRCODE_BASE,
QL_DATACALL_CFW_ATTACH_STATUS_GET_ERR,
QL_DATACALL_SEMAPHORE_CREATE_ERR,
QL_DATACALL_SEMAPHORE_TIMEOUT_ERR,
QL_DATACALL_CFW_ATTACH_REQUEST_ERR,
QL_DATACALL_CFW_ACTIVE_REQUEST_ERR    = 20 | QL_DATACALL_ERRCODE_BASE,
QL_DATACALL_ACTIVE_FAIL_ERR,
QL_DATACALL_CFW_DEACTIVE_REQUEST_ERR,
QL_DATACALL_NO_DFTPDN_CFG_CONTEXT,
QL_DATACALL_NO_DFTPDN_INFO_CONTEXT,
}ql_datacall_errcode_e;

```

● Member

Member	Description
<code>QL_DATACALL_SUCCESS</code>	Successful execution.
<code>QL_DATACALL_EXECUTE_ERR</code>	Failed execution.
<code>QL_DATACALL_MEM_ADDR_NULL_ERR</code>	Parameter address is NULL.
<code>QL_DATACALL_INVALID_PARAM_ERR</code>	Invalid parameter.
<code>QL_DATACALL_NW_REGISTER_TIMEOUT_ERR</code>	Network registration timeout.
<code>QL_DATACALL_CFW_ACT_STATE_GET_ERR</code>	Failed to get the status of PDP context activation.
<code>QL_DATACALL_REPEAT_ACTIVE_ERR</code>	Repeatedly activate PDP context ID.
<code>QL_DATACALL_REPEAT_DEACTIVE_ERR</code>	Repeatedly deactivate PDP context ID.
<code>QL_DATACALL_CFW_PDP_CTX_SET_ERR</code>	PDP context ID setting error.
<code>QL_DATACALL_CFW_PDP_CTX_GET_ERR</code>	PDP context ID getting error.
<code>QL_DATACALL_CS_CALL_ERR</code>	Data service operation failure due to an ongoing call.

<code>QL_DATACALL_CFW_CFUN_GET_ERR</code>	Failed to get functionality level.
<code>QL_DATACALL_CFUN_DISABLE_ERR</code>	Data service operation failure due to non-full-functionality.
<code>QL_DATACALL_NW_STATUS_GET_ERR</code>	Failed to get network registration status.
<code>QL_DATACALL_NOT_REGISTERED_ERR</code>	Network unregistered.
<code>QL_DATACALL_NO_MEM_ERR</code>	Failed to request memory.
<code>QL_DATACALL_CFW_ATTACH_STATUS_GET_ERR</code>	Failed to get network attachment status.
<code>QL_DATACALL_SEMAPHORE_CREATE_ERR</code>	Failed to create semaphore.
<code>QL_DATACALL_SEMAPHORE_TIMEOUT_ERR</code>	Waiting for semaphore timeout.
<code>QL_DATACALL_CFW_ATTACH_REQUEST_ERR</code>	Network attachment is rejected.
<code>QL_DATACALL_CFW_ACTIVE_REQUEST_ERR</code>	PDP context activation is rejected.
<code>QL_DATACALL_ACTIVE_FAIL_ERR</code>	Failed to activate PDP context.
<code>QL_DATACALL_CFW_DEACTIVE_REQUEST_ERR</code>	PDP context deactivation is rejected.
<code>QL_DATACALL_NO_DFTPDN_CFG_CONTEXT</code>	The default bearing context is not configured.
<code>QL_DATACALL_NO_DFTPDN_INFO_CONTEXT</code>	No default bearing context information.

NOTE

For the EC200D series module, the error code is subject to `ql_datacall_errcode_e` in `ql_api_datacall.h`.

3.3.2. `ql_set_data_call_asyn_mode`

This function sets the execution mode of `ql_start_data_call()` (see **Chapter 3.3.3** for details) and `ql_stop_data_call()` (see **Chapter 3.3.4** for details). There are two execution modes: synchronous and asynchronous.

- **Prototype**

```
ql_datacall_errcode_e ql_set_data_call_asyn_mode(uint8_t nSim, int profile_idx, bool enable)
```

- **Parameter**

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

profile_idx:

[In] PDP context ID. Range: 1–7.

enable:

[In] Execution mode.

- 0 Synchronous mode
- 1 Asynchronous mode

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.3. ql_start_data_call

This function starts a data call. Default execution mode: synchronous mode. To set the execution mode to asynchronous mode, call *ql_set_data_call_asyn_mode()* (see **Chapter 3.3.2** for details). The differences between synchronous mode and asynchronous mode are as follows:

Synchronous mode: The data call result code is returned after the function is executed. *QL_DATACALL_SUCCESS* being returned indicates that the data call is successful, IP address is obtained, and socket communication can be carried out.

Asynchronous mode: The function execution result code is returned after the function is executed. *QL_DATACALL_SUCCESS* being returned does not indicate a successful data call, but only a successful function execution. The registered callback function will inform the upper layer whether the data call is successful via *QUEC_DATACALL_ACT_RSP_IND* event. See **Chapter 3.3.6.1** for detailed information on the callback function and event.

● Prototype

```
ql_datacall_errcode_e ql_start_data_call
(
    uint8_t nSim,
    int profile_idx,
    int ip_version,
    char *apn_name,
    char *username,
    char *password,
    int auth_type
)
```

- **Parameter**

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

profile_idx:

[In] PDP context ID. Range: 1–7.

ip_version:

[In] IP version.

- 1 IPv4
- 2 IPv6
- 3 IPv4v6

apn_name:

[In] Access point name.

username:

[In] Username.

password:

[In] Password.

auth_type:

[In] Authentication type. See **Chapter 3.3.3.1** for details.

- **Return Value**

See **Chapter 3.3.1.1** for details.

NOTE

If VoLTE function is enabled, only the PDP context 1–5 can be available during a data call.

3.3.3.1. ql_pdp_auth_type_e

The enumeration of authentication type is defined as below:

```
typedef enum
{
    QL_PDP_AUTH_TYPE_NONE = 0,
    QL_PDP_AUTH_TYPE_PAP,
    QL_PDP_AUTH_TYPE_CHAP,
    QL_PDP_AUTH_TYPE_PAP_OR_CHAP,
}ql_pdp_auth_type_e;
```

● Member

Member	Description
<i>QL_PDP_AUTH_TYPE_NONE</i>	No authentication protocols.
<i>QL_PDP_AUTH_TYPE_PAP</i>	PAP.
<i>QL_PDP_AUTH_TYPE_CHAP</i>	CHAP.
<i>QL_PDP_AUTH_TYPE_PAP_OR_CHAP</i>	PAP or CHAP automatic authentication.

NOTE

For EC200D series and EC600U series modules, *ql_start_data_call()* does not support *QL_PDP_AUTH_TYPE_PAP_OR_CHAP*. If you need automatic authentication, refer to *nw_demo.c* and activate PDP again based on the reject cause returned by *QUEC_NW_CAUSE_IND* event in the callback.

3.3.4. ql_stop_data_call

This function stops a data call. The default execution mode is synchronous mode and you can set the execution mode to asynchronous mode by calling *ql_set_data_call_asyn_mode()* (see **Chapter 3.3.2** for details).

● Prototype

```
ql_datacall_errcode_e ql_stop_data_call(uint8_t nSim, int profile_idx)
```

● Parameter

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

profile_idx:

[In] PDP context ID. Range: 1–7.

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.5. ql_get_data_call_info

This function gets data call information.

● Prototype

```
ql_datacall_errcode_e ql_get_data_call_info(uint8_t nSim, int profile_idx, ql_data_call_info_s *info)
```

● Parameter

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

profile_idx:

[In] PDP context ID. Range: 1–7.

info:

[Out] Data call information. See **Chapter 3.3.5.1** for details.

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.5.1. ql_data_call_info_s

The structure of data call information is defined as below:

```
typedef struct
{
    int profile_idx;
    int ip_version;
    struct v4_info v4;
    struct v6_info v6;
}ql_data_call_info_s;
```

● Parameter

Type	Parameter	Description
int	<i>profile_idx</i>	PDP context ID. Range: 1–7.
int	<i>ip_version</i>	IP version. 1 IPv4 2 IPv6 3 IPv4v6
<i>struct v4_info</i>	<i>v4</i>	IPv4 information. See Chapter 3.3.5.2 for details.
<i>struct v6_info</i>	<i>v6</i>	IPv6 information. See Chapter 3.3.5.4 for details.

3.3.5.2. struct v4_info

The structure of IPv4 information is defined as below:

```
struct v4_info
{
    int state;
    struct v4_address_status addr;
};
```

● Parameter

Type	Parameter	Description
int	<i>state</i>	Data call status. 0 No data call 1 Successful data call
<i>struct v4_address_status</i>	<i>addr</i>	IPv4 address information. See Chapter 3.3.5.3 for details.

3.3.5.3. struct v4_address_status

The structure of IPv4 address information is defined as below:

```
struct v4_address_status
{
    ip4_addr_t ip;
    ip4_addr_t pri_dns;
    ip4_addr_t sec_dns;
};
```

● Parameter

Type	Parameter	Description
<i>ip4_addr_t</i>	<i>ip</i>	IPv4 address obtained. See Chapter 3.3.5.3.1 for details.
<i>ip4_addr_t</i>	<i>pri_dns</i>	Primary DNS IPv4 address. See Chapter 3.3.5.3.1 for details.
<i>ip4_addr_t</i>	<i>sec_dns</i>	Secondary DNS IPv4 address. See Chapter 3.3.5.3.1 for details.

3.3.5.3.1. ip4_addr_t

The structure of IPv4 address is defined as below:

```
struct ip4_addr_t
{
    uint32_t addr;
};
```

● Parameter

Type	Parameter	Description
<i>uint32_t</i>	<i>addr</i>	IPv4 address.

3.3.5.4. struct v6_info

The structure of IPv6 information is defined as below:

```
struct v6_info
{
    int state;
    struct v6_address_status addr;
};
```

- Parameter

Type	Parameter	Description
int	<i>state</i>	Data call status. 0 No data call 1 Successful data call
<i>struct v6_address_status</i>	<i>addr</i>	IPv6 address information. See Chapter 3.3.5.5 for details.

3.3.5.5. struct v6_address_status

The structure of IPv6 address information is defined as below:

```
struct v6_address_status
{
    ip6_addr_t ip;
    ip6_addr_t pri_dns;
    ip6_addr_t sec_dns;
};
```

- Parameter

Type	Parameter	Description
<i>ip6_addr_t</i>	<i>ip</i>	IPv6 address obtained. See Chapter 3.3.5.5.1 for details.
<i>ip6_addr_t</i>	<i>pri_dns</i>	Primary DNS IPv6 address. See Chapter 3.3.5.5.1 for details.
<i>ip6_addr_t</i>	<i>sec_dns</i>	Secondary DNS IPv6 address. See Chapter 3.3.5.5.1 for details.

3.3.5.5.1. ip6_addr_t

The structure of IPv6 address is defined as below:

```
struct ip6_addr_t
{
    uint32_t addr[4];
};
```

- Parameter

Type	Parameter	Description
uint32_t	<i>addr[4]</i>	IPv6 address.

3.3.6. ql_datacall_register_cb

This function registers the callback function of data call. Whether in asynchronous or synchronous mode, the module needs to register callback function of data call to report the event of deactivation or de-attachment from the network, and the module can initiate the data call process again after receiving this event.

● Prototype

```
ql_datacall_errcode_e ql_datacall_register_cb(uint8_t nSim, int profile_idx, ql_datacall_callback
datacall_cb, void *ctx)
```

● Parameter

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

profile_idx:

[In] PDP context ID. Range: 1–7.

datacall_cb:

[In] Callback function to be registered. See **Chapter 3.3.6.1** for details.

ctx:

[In] Pointer to the passed parameters of the callback function.

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.6.1. ql_datacall_callback

This callback function reports data call event.

● Prototype

```
typedef void (*ql_datacall_callback)(uint8_t nSim, unsigned int ind_type, int profile_idx, bool result, void
*ctx)
```

● Parameter

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

ind_type:

[In] Data call event type.

<code>QUEC_DATACALL_ACT_RSP_IND</code>	Response event of PDP activation result in asynchronous mode
<code>QUEC_DATACALL_DEACT_RSP_IND</code>	Response event of PDP deactivation result in asynchronous mode
<code>QUEC_DATACALL_PDP_DEACTIVE_IND</code>	Event of PDP deactivation or de-attachment by the network

profile_idx:

[In] PDP context ID. Range: 1–7.

result:

[In] Result of PDP context activation and deactivation.

- 0 Failed
- 1 Successful

ctx:

[In] Pointer to the passed parameters of the callback function.

● Return Value

None

3.3.7. ql_datacall_unregister_cb

This function cancels the callback function registered with `ql_datacall_register_cb()`. The callback function no longer receives data call events once it is cancelled.

● Prototype

```
ql_datacall_errcode_e ql_datacall_unregister_cb(uint8_t nSim, int profile_idx, ql_datacall_callback datacall_cb, void *ctx)
```

● Parameter

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

profile_idx:

[In] PDP context ID. Range: 1–7.

datacall_cb:

[In] Callback function to be cancelled. See **Chapter 3.3.6.1** for details.

ctx:

[In] Pointer to the passed parameters of the callback function.

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.8. ql_datacall_get_sim_profile_is_active

This function gets the current status of PDP context activation.

● Prototype

```
bool ql_datacall_get_sim_profile_is_active(uint8_t nSim, int profile_idx)
```

● Parameter

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

profile_idx:

[In] PDP context ID. Range: 1–7.

● Return Value

- 0 Not activated
- 1 Activated

3.3.9. ql_bind_sim_and_profile

This function binds the (U)SIM card in use and PDP context ID, and gets *sim_cid*, the combination of (U)SIM card and PDP context ID.

● Prototype

```
ql_datacall_errcode_e ql_bind_sim_and_profile(uint8_t nSim, int profile_idx, uint16_t *sim_cid)
```

● Parameter

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

profile_idx:

[In] PDP context ID. Range: 1–7.

sim_cid:

[Out] Combination of (U)SIM card and PDP context ID. Calculation formula: $nSim \ll 5 \mid profile_idx$

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.10. ql_get_sim_and_profile

This function gets (U)SIM card and PDP context ID from *sim_cid*.

● Prototype

```
ql_datacall_errcode_e ql_get_sim_and_profile(uint16_t sim_cid, uint8_t *nSim, int *profile_idx)
```

● Parameter

sim_cid:

[In] Combination of (U)SIM card and PDP context ID.

nSim:

[Out] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to

- 0.
- 0 (U)SIM card 1
- 1 (U)SIM card 2

profile_idx:

[Out] PDP context ID. Range: 1–7.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.11. ql_datacall_set_nat

This function enables NAT. The configuration takes effect after the module is rebooted.

- **Prototype**

```
ql_datacall_errcode_e ql_datacall_set_nat(uint32_t sim_profile_list)
```

- **Parameter**

sim_profile_list:

[In] List of (U)SIM card and PDP context ID combinations. The lower 16 bits belong to (U)SIM card 1 and the higher 16 bits belong to (U)SIM card 2. Each bit corresponds to a PDP context ID.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.12. ql_datacall_get_nat

This function gets the NAT-enabled (U)SIM card and PDP context ID.

- **Prototype**

```
ql_datacall_errcode_e ql_datacall_get_nat(uint32_t *sim_profile_list)
```

- **Parameter**

sim_profile_list:

[Out] List of (U)SIM card and PDP context ID combinations. The lower 16 bits belong to (U)SIM card 1 and the higher 16 bits belong to (U)SIM card 2. Each bit corresponds to a PDP context ID.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.13. ql_datacall_set_default_pdn_cfg

This function sets the default bearer configuration.

● Prototype

```
ql_datacall_errcode_e ql_datacall_set_default_pdn_cfg(uint8_t nSim, ql_data_call_default_pdn_cfg_s* ctx)
```

● Parameter

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

ctx:

[In] Configuration information. See **Chapter 3.3.13.1** for details.

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.13.1. ql_data_call_default_pdn_cfg_s

The structure of the default bearer configuration is defined as below:

```
typedef struct
{
    int ip_version;
    char apn_name[APN_LEN_MAX];
    char username[USERNAME_LEN_MAX];
    char password[PASSWORD_LEN_MAX];
    int auth_type;
}ql_data_call_default_pdn_cfg_s
```

● Parameter

Type	Parameter	Description
<i>int</i>	<i>ip_version</i>	IP type. 1 IPv4 2 IPv6 3 IPv4v6
<i>char</i>	<i>apn_name</i>	APN name.
<i>char</i>	<i>username</i>	User name.

<i>char</i>	password	Password.
<i>Int</i>	auth_type	Authentication type. See Chapter 3.3.3.1 for details.

3.3.14. ql_datacall_get_default_pdn_cfg

The function gets the default bearer configuration.

● Prototype

```
ql_datacall_errcode_e ql_datacall_get_default_pdn_cfg(uint8_t nSim, ql_data_call_default_pdn_cfg_s* ctx)
```

● Parameter

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

ctx:

[Out] Configuration information. See **Chapter 3.3.13.1** for details.

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.15. ql_datacall_get_default_pdn_info

The function gets the default bearer information.

● Prototype

```
ql_datacall_errcode_e ql_datacall_get_default_pdn_info(uint8_t nSim, ql_data_call_default_pdn_info_s* ctx)
```

● Parameter

nSim:

[In] (U)SIM card in use. If the module only supports one (U)SIM interface, this parameter must be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

ctx:

[Out] The bearer information. See **Chapter 3.3.15.1** for details.

- Return Value

See **Chapter 3.3.1.1** for details.

3.3.15.1. ql_data_call_default_pdn_info_s

The structure of the default bearer information is defined as below:

```
typedef struct
{
    uint8_t      ip_version;
    ip4_addr_t   ipv4;
    ip6_addr_t   ipv6;
    char         apn_name[APN_LEN_MAX];
}ql_data_call_default_pdn_info_s
```

- Parameter

Type	Parameter	Description
<i>uint8_t</i>	<i>ip_version</i>	IP type. 1 IPv4 2 IPv6 3 IPv4v6
<i>ip4_addr_t</i>	<i>ipv4</i>	IPv4 information. See Chapter 3.3.5.2 for details.
<i>ip6_addr_t</i>	<i>ipv6</i>	IPv6 information. See Chapter 3.3.5.4 for details.
<i>char</i>	<i>apn_name</i>	APN name.

4 Appendix References

Table 3: Related Documents

Document Name
[1] Quectel_ECx00G&EC600U&EG800G_Series_QuecOpen_CSDK_Quick_Start_Guide
[2] Quectel_EC200D_Series_QuecOpen_CSDK_Quick_Start_Guide
[3] Quectel_EC200D&ECx00G&EC600U&EG800G_Series_QuecOpen_Cellular_Network_Information_API_Reference_Manual

Table 4: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
CHAP	Challenge Handshake Authentication Protocol
DNS	Domain Name Server
ID	Identifier
IPv4	Internet Protocol Version 4
IPv6	Internet Protocol Version 6
NAT	Network Address Translation
PAP	Password Authentication Protocol
PDP	Packet Data Protocol
SDK	Software Development Kit
(U)SIM	(Universal) Subscriber Identity Module